



Universidade do Minho
Escola de Engenharia

Relatório Trabalho Prático SO

Tomé Francisco Amorim Moreira - a111907

Miguel Pedro Ribeiro Correia - a111773

José Pedro Fernandes da Costa - a109807

10 de maio de 2026

Índice

1	Arquitetura de Processos	2
1.1	Processo <i>controller</i>	2
1.2	Processos <i>runner</i>	2
2	Funcionalidades	2
2.1	Submissão de Comandos	2
2.2	Registo de Execução e Métricas de Desempenho	2
2.3	Processamento de vários comandos em paralelo	3
2.4	Seleção de Política de Escalonamento	3
2.5	Consulta do Estado do Sistema	3
2.6	Encerramento Controlado	3
2.7	Suporte a Redirecionamento e Pipes	3
3	Mecanismos de comunicação entre processos	4
3.1	FIFO Global	4
3.2	FIFOs Individuais	4
3.3	Justificação da Solução Adotada	5
3.4	Fluxo de Execução	5
3.5	Limitações dos Pipes	6
4	Políticas de Escalonamento Desenvolvidas	6
4.1	First Come First Served - FCFS	6
4.2	RR - User-based	6
4.3	Vantagens e Desvantagens das Abordagens	7
5	Avaliação de Desempenho e Escalonamento	7
5.1	Análise de Justiça e Interatividade	8
5.2	Impacto da Ordem de Submissão	8
5.3	Impacto do Grau de Multiprogramação	9
6	Conclusão	10

1 Arquitetura de Processos

O sistema é composto por um processo central, designado por *controller*, e por vários processos *runner*. Esta organização permite separar a gestão global do sistema da interação com os utilizadores, promovendo uma estrutura modular e escalável.

O *controller* assume a coordenação de toda a execução, enquanto os *runners* funcionam como pontos de entrada para as diferentes operações disponibilizadas pelo sistema.

1.1 Processo *controller*

O *controller* constitui o núcleo do sistema, sendo responsável pela gestão de todos os pedidos submetidos. Compete-lhe manter o estado global da execução, gerir as filas de processos, selecionar quais podem ser executados em cada instante e aplicar a política de escalonamento definida. Cabe-lhe ainda garantir que o número de processos em execução simultânea não ultrapassa o limite configurado, bem como assegurar um encerramento controlado do sistema.

1.2 Processos *runner*

Cada processo *runner* corresponde a uma interação individual com o sistema. Dependendo da operação pretendida, pode ser utilizado para submeter novos comandos, consultar o estado atual das tarefas ou solicitar o encerramento do *controller*.

Para além disso, o *runner* é responsável pela interação direta com o utilizador, apresentando informações sobre o estado das operações, nomeadamente a submissão de comandos, o início da execução, a conclusão das tarefas e os resultados das consultas efetuadas.

2 Funcionalidades

O sistema desenvolvido disponibiliza um conjunto de funcionalidades orientadas à submissão, gestão e monitorização de processos, permitindo controlar a execução concorrente de comandos segundo diferentes políticas de escalonamento.

2.1 Submissão de Comandos

Os utilizadores podem submeter novos comandos através do processo *runner*, utilizando o modo de execução apropriado. Cada pedido recebe um identificador único associado ao PID do processo *runner* que o originou.

Após a submissão, o pedido é encaminhado para o *controller*, que o coloca em fila de espera até existirem recursos disponíveis.

2.2 Registo de Execução e Métricas de Desempenho

O sistema regista informação detalhada sobre cada processo executado, armazenando os dados num ficheiro de log.

Para cada processo são registadas métricas relevantes, incluindo:

- *Turnaround time* — tempo total desde a submissão até à conclusão;
- *Response time* — tempo decorrido até ao início da execução;

- identificador do processo e utilizador associado;

Este registo permite analisar o comportamento do sistema e comparar o desempenho das diferentes políticas de escalonamento implementadas.

A recolha destas métricas revelou-se especialmente útil na fase de avaliação experimental, permitindo uma análise mais detalhada da justiça (*fairness*) e eficiência do escalonador.

2.3 Processamento de vários comandos em paralelo

O sistema permite a execução simultânea de múltiplos processos, respeitando sempre o limite máximo de paralelismo definido no arranque do controller.

Esta funcionalidade simula um ambiente multiprogramado, evitando sobrecarga excessiva do sistema e permitindo testar diferentes níveis de concorrência.

2.4 Seleção de Política de Escalonamento

No momento de inicialização do *controller*, é possível escolher a política de escalonamento a aplicar aos processos em espera.

Foram implementadas duas abordagens distintas: FCFS e Round Robin baseado em justiça entre utilizadores.

2.5 Consulta do Estado do Sistema

Através do modo de consulta, qualquer utilizador pode obter informação sobre o estado atual do sistema, consultando processos em execução e processos em espera (já escalonados de acordo com a ordem prevista de submissão).

Esta funcionalidade facilita a monitorização da atividade do escalonador.

2.6 Encerramento Controlado

O sistema suporta um mecanismo de encerramento ordenado. Após receber um pedido de *shutdown*, o controller deixa de aceitar novas submissões e aguarda pela conclusão dos processos pendentes.

Quando todas as tarefas terminam, o sistema encerra de forma segura.

2.7 Suporte a Redirecionamento e Pipes

O sistema suporta a execução de comandos que utilizam operadores de redirecionamento e composição através de pipes, nomeadamente os operadores `>`, `2>`, `<` e `|`.

Esta funcionalidade permite que os comandos submetidos pelos utilizadores apresentem um comportamento equivalente ao de uma bash tradicional, possibilitando:

- redirecionamento da saída standard para ficheiros;
- redirecionamento da saída de erro;
- leitura de dados a partir de ficheiros;
- encadeamento de múltiplos comandos através de pipes.

A interpretação e execução destes operadores é assegurada pelo processo *runner*. Desta forma, o sistema mantém transparência na execução dos comandos, permitindo ao utilizador utilizar expressões complexas sem necessidade de adaptações adicionais.

3 Mecanismos de comunicação entre processos

A comunicação entre os vários processos do sistema foi implementada recorrendo a *pipes com nome* (*named pipes* ou FIFOs). Esta solução permite a troca de informação entre processos independentes de forma simples e eficiente, sem recorrer a estratégias como a utilização de memória partilhada.

3.1 FIFO Global

O FIFO global funciona como ponto central de receção de pedidos por parte do *controller*. Todos os processos *runner* utilizam este canal para enviar mensagens como:

- submissão de novos comandos;
- notificação de conclusão de execução;
- pedidos de consulta de estado;
- pedidos de encerramento do sistema.

A utilização de um canal global apresenta várias vantagens. Em primeiro lugar, simplifica a arquitetura, uma vez que o *controller* necessita apenas de monitorizar um único ponto de entrada para receber todos os pedidos. Em segundo lugar, centraliza o fluxo de comunicação, permitindo uma gestão mais controlada e consistente dos pedidos recebidos.

3.2 FIFOs Individuais

Para além do FIFO global, cada processo *runner* cria um FIFO individual identificado pelo seu PID, garantindo unicidade. Estes canais são utilizados pelo *controller* para responder especificamente ao processo que originou o pedido.

Através destes FIFOs individuais são enviadas mensagens como:

- autorização para iniciar execução de um comando;
- listagem do estado atual do sistema;
- confirmação de conclusão do processo de shutdown.

Esta estratégia evita que respostas destinadas a diferentes processos sejam misturadas no mesmo canal de comunicação. Cada *runner* permanece bloqueado na leitura do seu FIFO dedicado até receber resposta, garantindo isolamento entre mensagens e simplificando o tratamento da informação recebida.

3.3 Justificação da Solução Adotada

A combinação entre um FIFO global e múltiplos FIFOs individuais revelou-se adequada ao problema proposto.

O FIFO global permite um modelo de comunicação muitos-para-um, ideal para submissão concorrente de pedidos por múltiplos utilizadores. Por outro lado, os FIFOs individuais implementam um modelo um-para-um, adequado ao envio de respostas personalizadas. Desta forma, obtém-se uma arquitetura modular e organizada, onde:

- os pedidos entram por um canal centralizado;
- as respostas são devolvidas de forma privada;
- o *controller* mantém controlo total sobre o escalonamento;
- reduz-se a interferência entre processos clientes na receção de respostas.

3.4 Fluxo de Execução

Como ilustrado na Figura 1, o funcionamento geral do sistema segue os seguintes passos:

1. o *runner* cria o seu FIFO individual;
2. envia um pedido ao *controller* através do FIFO global;
3. o *controller* recebe o pedido e decide quando o executar;
4. quando autorizado, envia a resposta pelo FIFO individual do *runner*;
5. após a execução, o *runner* comunica a conclusão novamente pelo FIFO global.

Esta separação entre canal de pedidos e canal de respostas permite uma comunicação clara, previsível e facilmente extensível.

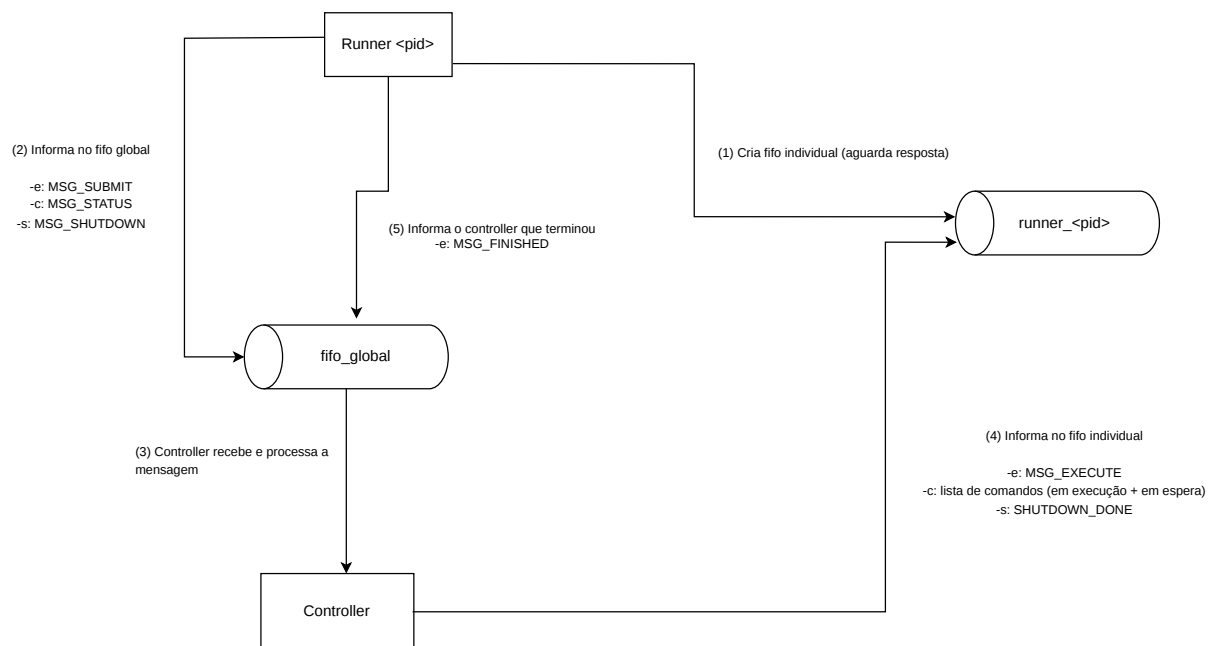


Figura 1: Mecanismos de comunicação entre runners e controller

3.5 Limitações dos Pipes

A utilização de FIFOs introduz algumas limitações inerentes ao mecanismo de comunicação adotado, nomeadamente no que diz respeito à capacidade limitada dos buffers dos pipes.

Estes podem originar bloqueios quando um processo tenta escrever grandes volumes de dados sem que exista leitura simultânea do outro lado. No entanto, este cenário não se verificou no contexto do projeto.

Tal deve-se ao cuidado na estruturação da comunicação, garantindo que as operações de leitura e escrita ocorrem de forma coordenada, prevenindo situações de bloqueio mesmo em condições potencialmente mais exigentes.

4 Políticas de Escalonamento Desenvolvidas

Foram desenvolvidas duas políticas de escalonamento distintas, cada uma com objetivos e comportamentos diferentes relativamente à gestão dos processos submetidos ao sistema.

4.1 First Come First Served - FCFS

O algoritmo mais simples implementado foi o *First Come First Served* (FCFS). Nesta abordagem, os processos são executados pela mesma ordem em que chegam ao sistema. Assim, o primeiro processo a ser submetido será também o primeiro a ser executado, desde que existam recursos disponíveis.

Sempre que um processo termina, o próximo processo em espera é selecionado segundo essa mesma ordem de chegada.

Trata-se de uma política simples, previsível e fácil de implementar, sendo adequada para cenários onde a prioridade principal é respeitar a sequência temporal de submissão.

4.2 RR - User-based

Foi também desenvolvida uma versão adaptada do algoritmo *Round Robin* (RR), focada não na alternância temporal entre processos, mas sim à distribuição equilibrada de execuções entre utilizadores.

Ao contrário do Round Robin tradicional, onde os processos alternam por fatias de tempo, nesta abordagem a alternância é feita entre utilizadores. O objetivo principal é garantir *fairness*, evitando que um utilizador monopolize os recursos ao submeter muitos processos consecutivos.

De forma simplificada, o sistema percorre os utilizadores que possuem processos em espera e seleciona um processo de cada utilizador por vez, repetindo esse ciclo sucessivamente. Assim, mesmo que um utilizador tenha muitos processos pendentes, os restantes continuam a ter oportunidade de execução.

Por exemplo, se o utilizador A tiver 5 processos e o utilizador B tiver 1 processo, a execução poderá seguir uma ordem semelhante a:

$$A1 \rightarrow B1 \rightarrow A2 \rightarrow A3 \rightarrow A4 \rightarrow A5$$

Desta forma, o utilizador B não fica bloqueado atrás de todos os processos do utilizador A.

4.3 Vantagens e Desvantagens das Abordagens

FCFS

Vantagens:

- Implementação simples e eficiente;
- Ordem de execução previsível;
- Adequado quando os processos têm duração semelhante;
- Respeita rigorosamente a ordem de chegada.

Desvantagens:

- Pode gerar tempos de espera elevados caso processos longos cheguem primeiro;
- Um único utilizador pode ocupar a fila com vários processos;
- Menor sensação de justiça entre utilizadores.

RR User-based

Vantagens:

- Distribuição mais justa dos recursos entre utilizadores;
- Evita monopolização por parte de um único utilizador;
- Melhor resposta em ambientes multiutilizador;
- Todos os utilizadores obtêm oportunidade de execução.

Desvantagens:

- Implementação mais complexa;
- Pode aumentar ligeiramente o tempo total de execução;
- Nem sempre respeita a ordem global de chegada dos processos.

De forma geral, o FCFS é mais vantajoso em ambientes simples e com baixa concorrência, enquanto o RR baseado em utilizadores é mais adequado em sistemas partilhados, onde a justiça entre utilizadores é um fator relevante.

5 Avaliação de Desempenho e Escalonamento

Nesta fase, o sistema foi submetido a um conjunto de testes com o objetivo de avaliar o comportamento das políticas de escalonamento implementadas: *First-Come, First-Served* (FCFS) e *User-based Round Robin* (RR).

A análise incidiu principalmente sobre dois aspetos:

- A justiça (*fairness*) entre utilizadores;
- O impacto do grau de paralelismo (*Max_parallel*) no desempenho global.

5.1 Análise de Justiça e Interatividade

O teste `test_fairness.sh` foi concebido para simular um cenário onde um utilizador (User 1) submete uma carga intensiva de processos longos, seguido de pedidos curtos de outros utilizadores.

- **FCFS:** Como esperado, os processos são executados por ordem de chegada. Assim, os pedidos dos utilizadores 2, 3 e 4 ficam bloqueados atrás da carga do User 1, resultando em tempos de resposta significativamente mais elevados (aproximadamente 9 ms).
- **RR (User-based):** O escalonador alterna entre utilizadores, garantindo que todos têm oportunidade de executar processos. Mesmo sem interromper tarefas a meio (isto é, sem *time slice*), esta abordagem permite que processos curtos de outros utilizadores passem à frente na fila, reduzindo o tempo de resposta (valores inferiores a 1 ms).

Este teste demonstra claramente que o algoritmo RR melhora a interatividade e evita a monopolização do sistema por um único utilizador.

Métrica	FCFS	RR
Average Turnaround time (ms)	6.088	4.915
Average Response time (ms)	5.214	4.041

User	FCFS		RR	
	Turnaround	Response	Turnaround	Response
User 1	5.525	4.520	5.531	4.526
User 2	9.845	9.842	0.804	0.801
User 3	9.848	9.845	0.807	0.804
User 4	9.846	9.842	0.810	0.807

Tabela 1: Resultados do teste `test_fairness.sh` (Max_paralel = 2)

Os resultados apresentados nesta tabela evidenciam claramente o impacto do algoritmo de escalonamento na interatividade do sistema. No caso do FCFS, observa-se que os utilizadores 2, 3 e 4 apresentam tempos de resposta e turnaround significativamente elevados, uma vez que os seus pedidos ficam bloqueados atrás da carga intensiva submetida pelo utilizador 1.

Por outro lado, o algoritmo Round Robin demonstra um comportamento mais equilibrado. Apesar de o utilizador 1 manter tempos semelhantes, os restantes utilizadores beneficiam de tempos de resposta muito reduzidos, próximos de 0.8 ms. Isto deve-se ao facto de o escalonador alternar entre utilizadores, permitindo que processos curtos sejam executados sem terem de esperar pela conclusão de toda a carga inicial.

Este comportamento confirma que o RR melhora substancialmente a justiça (*fairness*) e a interatividade do sistema, evitando a monopolização dos recursos por um único utilizador.

5.2 Impacto da Ordem de Submissão

No teste `test_consc.sh`, os slots de execução foram inicialmente ocupados e, de seguida, o utilizador 1 submeteu vários processos curtos consecutivos antes da submissão de pro-

cessos longos pelos restantes utilizadores. Este cenário permite observar o impacto da ordem de chegada e da política de escalonamento no desempenho global do sistema.

- **FCFS:** Favorece os processos submetidos primeiro. Como o utilizador 1 preenche rapidamente a fila com tarefas curtas, os seus processos são executados quase imediatamente, enquanto os restantes utilizadores ficam bloqueados à espera da libertação dos recursos.
- **RR:** Introduce maior equilíbrio entre utilizadores através da alternância da execução. No entanto, essa alternância aumenta o tempo médio de turnaround e response, especialmente para tarefas curtas, que deixam de terminar imediatamente.

Este comportamento evidencia o compromisso entre eficiência e justiça (*fairness*) nas diferentes políticas de escalonamento.

Métrica	FCFS	RR
Average Turnaround time (ms)	2.765	8.147
Average Response time (ms)	0.742	6.124

User	FCFS		RR	
	Turnaround	Response	Turnaround	Response
User 1	0.078	0.059	7.430	7.410
User 2	0.146	0.038	0.113	0.000
User 3	20.163	10.153	10.122	0.113
User 4	10.153	0.144	20.130	10.120
User 5	10.156	0.148	10.120	0.112

Tabela 2: Resultados do teste `test_consc.sh` (Max_paralel = 2)

A análise dos resultados mostra que o algoritmo FCFS favorece significativamente os processos submetidos primeiro. Como o utilizador 1 submete várias tarefas curtas consecutivas, estas são executadas quase imediatamente, apresentando tempos de turnaround e response time extremamente reduzidos. Em contrapartida, os restantes utilizadores ficam à espera da execução de toda a fila já existente.

No algoritmo Round Robin, a alternância entre utilizadores reduz o favorecimento imediato dos primeiros processos submetidos e distribui os recursos de forma mais equilibrada. Contudo, esta política aumenta significativamente os tempos médios das tarefas curtas, uma vez que o escalonador alterna a ordem de seleção entre utilizadores, atrasando a execução imediata dos processos curtos.

Assim, este teste evidencia claramente o compromisso entre eficiência e justiça: o FCFS apresenta melhor desempenho médio neste cenário específico, enquanto o RR proporciona maior equilíbrio entre utilizadores.

5.3 Impacto do Grau de Multiprogramação

Para avaliar o impacto do paralelismo, foram realizados testes com diferentes valores de *Max_parallel*, variando entre 2 e 100. Os resultados encontram-se representados na Figura 2.

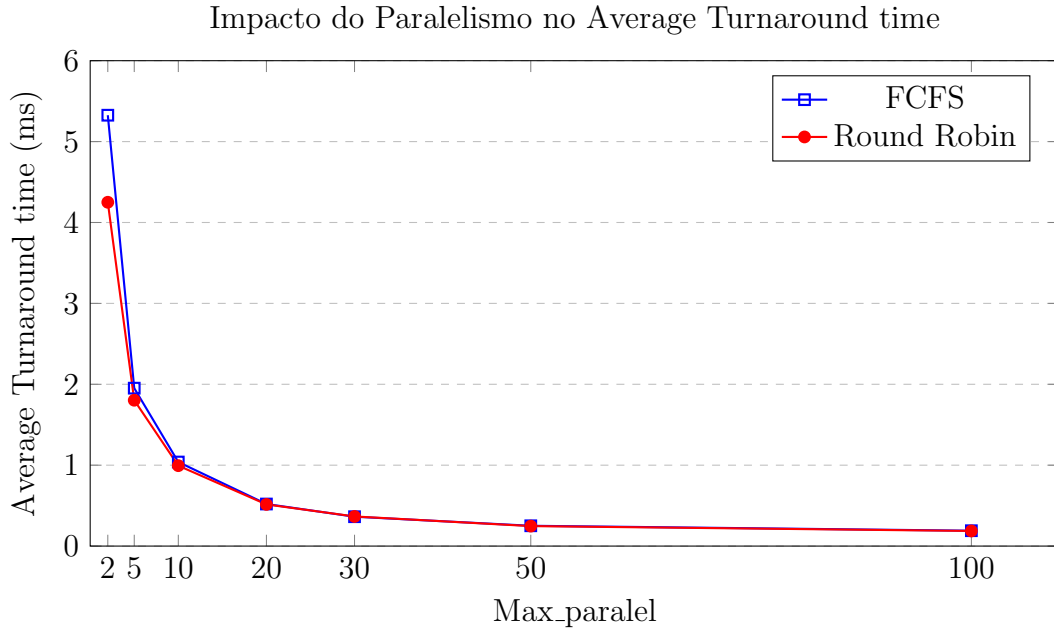


Figura 2: Relação entre o grau de paralelismo e o Average Turnaround time.

Observa-se uma redução significativa do Average Turnaround time à medida que o grau de paralelismo aumenta. Para valores baixos ($Max_paralel = 2$), o sistema encontra-se em forte contenção, sendo necessário recorrer a políticas de escalonamento eficientes. Neste cenário, o algoritmo Round Robin apresenta melhor desempenho médio, devido à distribuição mais equilibrada dos processos.

À medida que o valor de $Max_paralel$ aumenta, a diferença entre os algoritmos torna-se progressivamente irrelevante. Para valores elevados (por exemplo, $Max_paralel = 100$), ambos convergem para valores praticamente idênticos (cerca de 0.19 ms), uma vez que o sistema consegue executar quase todos os processos em simultâneo.

Os resultados obtidos dependem diretamente da natureza dos testes realizados. Diferentes cargas de trabalho poderiam produzir valores absolutos distintos. No entanto, a tendência observada mantém-se: quanto maior o paralelismo, menor o impacto do algoritmo de escalonamento.

6 Conclusão

O trabalho desenvolvido resultou num sistema funcional de gestão de processos concorrentes, capaz de lidar de forma controlada com múltiplos pedidos em simultâneo.

A separação entre componentes e a utilização de mecanismos de comunicação entre processos permitiram garantir um funcionamento organizado, evitando inconsistências e assegurando previsibilidade na execução.

A implementação das políticas de escalonamento e a análise do seu impacto reforçaram a importância destas decisões no comportamento global do sistema.

De forma geral, o sistema desenvolvido demonstra robustez e flexibilidade, respondendo de forma eficaz aos requisitos propostos e validando as decisões tomadas ao longo do desenvolvimento.